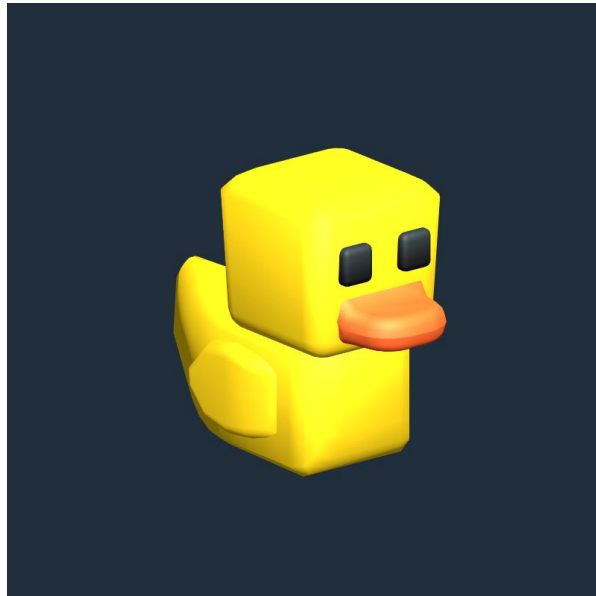


Guía técnica del código

Patitos en batalla



Documento complementario del GDD

Unity 2019.4.19f1 • C Sharp • Escena principal Main

1 Propósito y alcance

Este documento explica las partes del código que sostienen la versión modificada del juego. Su propósito es ayudar a presentar el proyecto, reconocer qué script controla cada comportamiento y facilitar cambios posteriores sin tener que leer todo el repositorio.

La conclusión técnica principal es que el personaje visual cambió de tanque a pato de goma, pero la arquitectura conserva los nombres originales Tank. El prefab sigue reuniendo movimiento, disparo, salud, física y UI; el modelo de pato funciona como su representación visual.

Mapa de responsabilidades

Script	Responsabilidad principal
GameManager.cs	Inicia la partida, crea jugadores y administra rondas y ganadores.
GameTimerPause.cs	Cuenta el tiempo, actualiza la interfaz y pausa o reanuda el juego.
TankManager.cs	Conecta cada personaje con su número, color, aparición y controles.
TankMovement.cs	Lee ejes de entrada y mueve o gira el Rigidbody.
TankShooting.cs	Carga el disparo, crea el proyectil y aplica su velocidad.
TankHealth.cs	Gestiona vida, barra de salud, muerte y explosión.
ShellExplosion.cs	Detecta impactos, calcula daño por distancia y aplica fuerza.

Flujo general

Main.unity → GameManager.Start → SpawnAllTanks → GameLoop → ronda → ganador

2 Inicio de la partida

No existe actualmente un botón de inicio independiente. Unity carga la escena Main y llama automáticamente a Start en GameManager. Ese método prepara las esperas, crea los personajes, registra los objetivos de cámara y comienza la corrutina que gobierna la partida.

Inicio automático en GameManager

```
private void Start()
{
    m_StartWait = new WaitForSeconds(m_StartDelay);
    m_EndWait = new WaitForSeconds(m_EndDelay);

    SpawnAllTanks();
    SetCameraTargets();
    StartCoroutine(GameLoop());
}
```

Fuente del proyecto Assets/Scripts/Managers/GameManager.cs líneas 24 a 34

Creación de los personajes

SpawnAllTanks crea una instancia por jugador

```
for (int i = 0; i < m_Tanks.Length; i++)
{
    m_Tanks[i].m_Instance = Instantiate(
        m_TankPrefab,
        m_Tanks[i].m_SpawnPoint.position,
        m_Tanks[i].m_SpawnPoint.rotation);
    m_Tanks[i].m_PlayerNumber = i + 1;
    m_Tanks[i].Setup();
}
```

Cada elemento de m_Tanks contiene un punto de aparición y un color. El mismo prefab se reutiliza para ambos jugadores; el número asignado decide qué entradas del teclado leerá cada instancia.

Qué sería un botón de inicio

Si se desea agregarlo en el futuro, el menú debe mantener Time.timeScale en cero o impedir el inicio de GameLoop hasta que el botón invoque un método público. Esa función no forma parte de la versión actual y conviene implementarla en un controlador de menú separado para no mezclarla con la lógica de rondas.

3 Temporizador y pausa

GameTimerPause es la mejora principal de interfaz. Awake restablece la escala temporal, enlaza el botón con TogglePause y dibuja el valor inicial. Update escucha Escape y aumenta el tiempo solo cuando el juego no está pausado.

Preparación y actualización del HUD

```
private void Awake()
{
    Time.timeScale = 1f;
    if (pauseButton != null)
        pauseButton.onClick.AddListener(TogglePause);
    UpdateDisplay();
}

private void Update()
{
    if (Input.GetKeyDown(KeyCode.Escape))
        TogglePause();

    if (!isPaused)
    {
        elapsedTime += Time.unscaledDeltaTime;
        UpdateDisplay();
    }
}
```

Fuente del proyecto Assets/Scripts/Managers/GameTimerPause.cs líneas 13 a 30

Cómo se detiene el juego

Un mismo método atiende al botón y a Escape

```
public void TogglePause()
{
    isPaused = !isPaused;
    Time.timeScale = isPaused ? 0f : 1f;
    if (pauseButtonText != null)
        pauseButtonText.text = isPaused ? "REANUDAR" : "PAUSAR";
}
```

Time.timeScale controla la velocidad del tiempo simulado. Al usar 0, la física y los métodos dependientes de deltaTime se detienen. El texto del botón comunica el siguiente estado disponible.

4 Lectura y presentación del tiempo

Conversión de segundos al formato minutos y segundos

```
private void UpdateDisplay()
{
    if (timerText == null) return;
    int totalSeconds = Mathf.FloorToInt(elapsedTime);
    timerText.text = string.Format(
        "TIEMPO {0:00}:{1:00}",
        totalSeconds / 60,
        totalSeconds % 60);
}
```

Fuente del proyecto Assets/Scripts/Managers/GameTimerPause.cs líneas 41 a 46

elapsedTime almacena segundos como float. FloorToInt descarta la fracción para mostrar segundos completos. La división entera calcula minutos y el módulo obtiene los segundos restantes. El formato 00 asegura dos dígitos, por ejemplo 02:07.

Por qué usa unscaledDeltaTime

Time.unscaledDeltaTime mide el tiempo real entre cuadros sin multiplicarlo por Time.timeScale. En este proyecto el contador solo se incrementa dentro de if (!isPaused), de modo que se detiene durante la pausa por decisión explícita. Esta combinación evita depender accidentalmente del estado global del motor.

Protecciones incluidas

- Las referencias timerText y pauseButtonText se validan antes de usarse, lo que evita errores NullReferenceException si falta un enlace en el Inspector.
- OnDestroy restablece Time.timeScale a 1 para que otra escena o una nueva ejecución no quede congelada.
- El mismo TogglePause se reutiliza para el clic y la tecla Escape, por lo que ambos caminos producen exactamente el mismo estado.

5 Controles de los jugadores

Acción	Jugador 1	Jugador 2	Entrada lógica
Avanzar y retroceder	W y S	Flechas arriba y abajo	Vertical1 o Vertical2
Girar	A y D	Flechas izquierda y derecha	Horizontal1 o Horizontal2
Cargar y disparar	Espacio	Enter	Fire1 o Fire2
Pausar o reanudar	Escape o botón	Escape o botón	KeyCode Escape

Movimiento

El número del jugador selecciona sus ejes

```
m_MovementAxisName = "Vertical" + m_PlayerNumber;
m_TurnAxisName = "Horizontal" + m_PlayerNumber;

m_MovementInputValue = Input.GetAxis(m_MovementAxisName);
m_TurnInputValue = Input.GetAxis(m_TurnAxisName);
```

La física recibe desplazamiento y rotación

```
Vector3 movement = transform.forward
    * m_MovementInputValue * m_Speed * Time.deltaTime;
m_Rigidbody.MovePosition(m_Rigidbody.position + movement);

float turn = m_TurnInputValue * m_TurnSpeed * Time.deltaTime;
Quaternion turnRotation = Quaternion.Euler(0f, turn, 0f);
m_Rigidbody.MoveRotation(m_Rigidbody.rotation * turnRotation);
```

Fuente del proyecto *Assets/Scripts/Tank/TankMovement.cs* líneas 42 a 55 y 96 a 115

6 Disparo cargado

El disparo no usa una fuerza fija. Al pulsar, se reinicia la carga; mientras la tecla permanece presionada, aumenta la fuerza y se actualiza el slider. Al soltar o alcanzar el máximo, Fire crea el proyectil.

Carga gradual del disparo

```
m_FireButton = "Fire" + m_PlayerNumber;
m_ChargeSpeed = (m_MaxLaunchForce - m_MinLaunchForce)
    / m_MaxChargeTime;

if (Input.GetButton(m_FireButton) && !m_Fired)
{
    m_CurrentLaunchForce += m_ChargeSpeed * Time.deltaTime;
    m_AimSlider.value = m_CurrentLaunchForce;
}
```

Creación y lanzamiento del proyectil

```
Rigidbody shellInstance = Instantiate(
    m_Shell,
    m_FireTransform.position,
    m_FireTransform.rotation) as Rigidbody;

shellInstance.velocity =
    m_CurrentLaunchForce * m_FireTransform.forward;
```

Fuente del proyecto Assets/Scripts/Tank/TankShooting.cs líneas 32 a 78 y 82 a 100

Relación con el modelo de pato

m_FireTransform es un objeto hijo colocado delante del personaje. En la versión de patitos se ajustó cerca del pico para que el proyectil parezca salir desde allí. La lógica de disparo no necesita conocer la forma del modelo; solo utiliza la posición y orientación de ese Transform.

7 Personajes y apariencia

Cada pato jugable nace del prefab Tank. El nombre técnico se conserva porque los scripts originales dependen de esa estructura. El modelo rubberDuck.fbx y el material DuckPlayer.mat sustituyen la apariencia, mientras que RigidBody, Collider, Canvas, TankMovement, TankShooting y TankHealth mantienen el comportamiento.

Configuración por jugador

TankManager conecta identidad, controles y color

```
m_Movement = m_Instance.GetComponent<TankMovement>();
m_Shooting = m_Instance.GetComponent<TankShooting>();

m_Movement.m_PlayerNumber = m_PlayerNumber;
m_Shooting.m_PlayerNumber = m_PlayerNumber;

MeshRenderer[] renderers =
    m_Instance.GetComponentsInChildren<MeshRenderer>();
for (int i = 0; i < renderers.Length; i++)
    renderers[i].material.color = m_PlayerColor;
```

Fuente del proyecto Assets/Scripts/Managers/TankManager.cs líneas 25 a 47

Activación durante las rondas

El administrador decide cuándo puede actuar cada personaje

```
public void DisableControl()
{
    m_Movement.enabled = false;
    m_Shooting.enabled = false;
    m_CanvasGameObject.SetActive(false);
}

public void EnableControl()
{
    m_Movement.enabled = true;
    m_Shooting.enabled = true;
    m_CanvasGameObject.SetActive(true);
}
```

Esto evita que un jugador se mueva durante los mensajes de inicio y final de ronda. Reset devuelve cada instancia a su punto de aparición y reactiva sus componentes para recuperar la vida inicial.

8 Vida impacto y eliminación

La salud centraliza daño y muerte

```
public void TakeDamage(float amount)
{
    m_CurrentHealth -= amount;
    SetHealthUI();

    if (m_CurrentHealth <= 0f && !m_Dead)
        OnDeath();
}
```

El daño disminuye con la distancia a la explosión

```
Vector3 explosionToTarget = targetPosition - transform.position;
float explosionDistance = explosionToTarget.magnitude;
float relativeDistance =
    (m_ExplosionRadius - explosionDistance) / m_ExplosionRadius;
float damage = relativeDistance * m_MaxDamage;
return Mathf.Max(0f, damage);
```

Fuente del proyecto Assets/Scripts/Tank/TankHealth.cs líneas 37 a 67

Fuente del proyecto Assets/Scripts/Shell/ShellExplosion.cs líneas 67 a 83

Cuando la vida llega a cero, el pato se desactiva, se reproducen partículas y sonido, y GameManager detecta que queda uno o ningún personaje activo. Ese resultado termina la ronda y actualiza el marcador de victorias.

Cadena de responsabilidades

TankShooting → ShellExplosion → TankHealth → GameManager

9 Ciclo de rondas

Secuencia principal de la partida

```
private IEnumerator GameLoop()
{
    yield return StartCoroutine(RoundStarting());
    yield return StartCoroutine(RoundPlaying());
    yield return StartCoroutine(RoundEnding());

    if (m_GameWinner != null)
        Application.LoadLevel(Application.loadedLevel);
    else
        StartCoroutine(GameLoop());
}
```

Fuente del proyecto *Assets/Scripts/Managers/GameManager.cs* líneas 71 a 93

Fase	Acción del código	Estado de los jugadores
RoundStarting	Reinicia posiciones, ajusta cámara, aumenta la ronda y espera.	Controles desactivados
RoundPlaying	Limpia el mensaje y espera hasta que quede como máximo un personaje.	Controles activados
RoundEnding	Calcula ganador, aumenta victorias y muestra resultados.	Controles desactivados

Las corrutinas permiten repartir la secuencia entre varios cuadros sin bloquear Unity. `WaitForSeconds` respeta `Time.timeScale`, por lo que una pausa también congela las esperas activas de las rondas.

10 Conexiones en la escena Main

El código funciona porque la escena y los prefabs enlazan referencias públicas desde el Inspector. Si una referencia cambia al reemplazar assets, conviene revisar esta lista antes de modificar scripts.

Objeto o componente	Referencias que deben estar asignadas
GameManager	CameraControl, MessageText, Tank prefab, lista de jugadores y SpawnPoints.
GameHUD con GameTimerPause	TimerText, PauseButton y texto interno del botón.
Prefab Tank con pato	Rigidbody, colliders, Canvas, FireTransform, AudioSources y scripts Tank.
TankShooting	Shell prefab, FireTransform, AimSlider y clips de carga y disparo.
TankHealth	Slider, FillImage y prefab de explosión.
Shell	Capa Players, partículas, AudioSource, radio y daño máximo.

Comprobación rápida

- Main aparece como escena principal y única escena necesaria en Build Settings.
- El temporizador empieza en 00:00 y se detiene al pausar.
- El botón alterna entre PAUSAR y REANUDAR; Escape produce el mismo resultado.
- Los dos patitos reciben colores diferentes y responden a sus respectivos controles.
- El proyectil nace junto al pico, causa daño y puede finalizar una ronda.

11 Mejoras recomendadas

Las siguientes mejoras son coherentes con la estructura actual y pueden presentarse como trabajo futuro. No forman parte del código implementado salvo que se indique lo contrario.

Prioridad	Mejora	Cambio técnico sugerido
Alta	Menú de inicio real	Crear MenuController y comenzar GameLoop mediante un método público después del clic.
Alta	Reinicio desde pausa	Añadir botón Reiniciar que restaure Time.timeScale antes de recargar Main.
Media	Tiempo por ronda	Reiniciar elapsedTime en RoundStarting o mantener contadores separados de partida y ronda.
Media	Nuevo Input System	Sustituir nombres de ejes por acciones y mapas para teclado y mandos.
Media	Nombres temáticos	Renombrar clases gradualmente a Duck sin romper referencias serializadas del prefab.
Baja	Indicador de estado	Mostrar PAUSA en el centro de la pantalla además de cambiar el texto del botón.

Archivos principales para la exposición

- Assets/Scripts/Managers/GameTimerPause.cs
- Assets/Scripts/Managers/GameManager.cs
- Assets/Scripts/Managers/TankManager.cs
- Assets/Scripts/Tank/TankMovement.cs
- Assets/Scripts/Tank/TankShooting.cs
- Assets/Scripts/Tank/TankHealth.cs
- Assets/Scripts/Shell/ShellExplosion.cs

Cierre

El proyecto separa correctamente la lógica general, la identidad de cada jugador y los comportamientos del personaje. La mejora de temporizador y pausa se integró sin alterar el sistema de rondas. Esta separación permite cambiar la apariencia del tanque por un pato y conservar las mecánicas centrales del juego.